MIGRATION HAND-OFF

Addressbook, Vaadin 8 to Vaadin 25

Hand-off document

DRAFT, WRITTEN BEFORE THE CUSTOMER'S ANSWERS ON THE OPEN DECISIONS (SECTION 7)

Version	Draft 1
Date	2026-10-22
Status	Draft, written before the customer's answers on the open decisions (section 7)
Prepared for	Product owner, operators and developers of the Addressbook app

The open decisions in section 7 are written on the recommended answers where one exists and are marked "pending the customer's answer". This document is regenerated from its Markdown source (docs/handoff.md) once they are answered.

Contents

1 **Executive summary** 2

2 **What changed** 3

2.1 For users 3

2.2 For operators 4

2.3 For developers 5

3 **Environment, build, run, test and data** 6

3.1 Production build and run 6

3.2 Development mode 6

3.3 Docker environment 6

3.4 Tests 6

3.5 CI 7

3.6 Database 7

4 **Known pre-existing issues** 8

5 **Old and new side by side** 9

6 **Possible follow-up work** 10

7 **Open decisions** 11

7.1 Behaviour differences to accept or reject 11

7.2 Where the migrated code ends up 11

7.3 Acceptance 11

1 Executive summary

Outcome. The Addressbook app now runs on Vaadin 25 (25.3.3, the free `vaadin-core` components only), Java 21 and Jetty 12.1. It has the same single screen as before: a filter field, a "New contact" button, the contact list, and a form with Save and Cancel that opens when a contact is selected. The data is still the in-memory demo list of 100 contacts. All 11 user workflows recorded on the old app run to the end on the new app. Eight give the same results, or differ only as already agreed; three show small differences that wait for your decision (section 7). The app is built, started, smoke-checked and tested end to end in CI on every push.

What changed.

- For users: a new look (Vaadin's Lumo theme instead of Valo) with the same layout, fields, buttons and data; a new calendar popup; small differences in messages and keyboard behaviour (section 2.1).
- For operators: a Jakarta EE 11 / Servlet 6.1 container (Jetty 12.1) on Java 21 instead of Jetty 9.2 on Java 8 bytecode; production mode is now decided when the war is built; a new set of dependencies, all Apache 2.0 (section 2.2).
- For developers: Vaadin Flow's Java API instead of the Vaadin 7 compatibility API; 7 unit tests and 10 end-to-end tests, where the old app had none (section 2.3).

Risks.

- commons-beanutils 1.9.2 stays, as agreed, with two known vulnerabilities (and one more in commons-collections 3.2.1, which it pulls in). The app only copies its own contact objects with it, so exposure is low. Removing it is a small follow-up (section 6).
- In production mode a server-side error is not visible to the user or in the browser console; only the server log shows it. CI fails on any error in the server log.
- The end-to-end tests wait on an internal Vaadin client object; a future Vaadin update can make them flaky without warning (section 6).

Decisions needed from you. Eight behaviour differences to accept or reject, one of them a choice between keeping a difference and taking a fix (section 7.1), and where the migrated code ends up in your repository (section 7.2).

2 What changed

This chapter lists only the changes recorded in the migration's change register. Where a change is a behaviour change you have accepted or still have to accept, it says so.

The migration started from your app on Vaadin 8.0.0.alpha2, a pre-release that is only available from a Vaadin pre-release repository. The first step was to bring it to Vaadin 8.14.5 (a free Vaadin 8 release from Maven Central) without other changes, and to add CI and a Docker environment. That stabilised Vaadin 8 app is the reference the new app was compared with. It is on the `main` branch of the output repository; the migrated app is on `mig-dev`.

2.1 For users

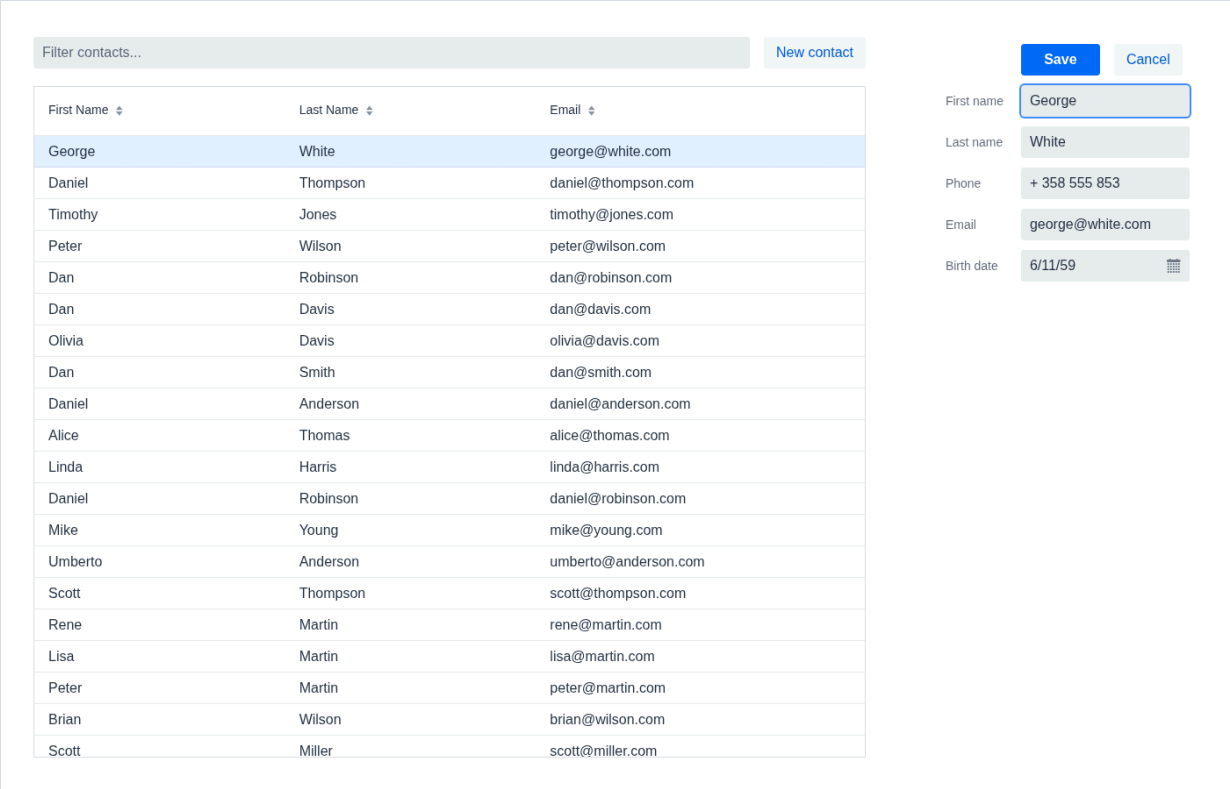


Figure 1. The new app with a contact open (Vaadin 25, Lumo theme), 1400 x 900 window.

Area	Old (Vaadin 8)	New (Vaadin 25)	Status
Look	Valo theme	Lumo theme: grey-filled fields, plain buttons, no grid column lines, pale row selection, system font, white background. Same layout: list on the left, a one-column form with labels to the left of the fields on the right, Save and Cancel above the fields, the same margins. About 21 rows visible at 1400 x 900 (old 19.5)	Theme chosen at the first milestone (Lumo)
Birth date in the alpha	Dates shown with a trailing comma, typed dates rejected	Fixed by the Vaadin 8.14.5 upgrade and kept: `6/11/59`, typed dates save	Old bug not copied

Area	Old (Vaadin 8)	New (Vaadin 25)	Status
Birth date field	`M/d/yy`, typed dates accepted	Same format and typing. A typed two-digit year in the boundary year is read by year, not by day (example: `10/22/46` typed in 2026 is stored as 1946, the old app stored 2046). The date order is always the US one, also for an en-GB browser	Pending your answer (section 7)
Birth date after Save	Saving an untouched contact kept the time of day of the demo data	Save stores the birth date at midnight, so text that matched a time of day in the filter no longer matches a saved contact	Accepted at the start of the migration
Invalid birth date	Red border and "!" marker; the message only as a tooltip	"Date format not recognized" under the field as soon as you leave it. Save does nothing in both	Pending your answer
Calendar popup	One month with arrows	Scrolling months with a year column, Today and Cancel buttons. Same function	Pending your answer
Calendar for a new contact	Opens on the current month	Opens on the month last shown in this browser tab (for example June 1959 after looking at George White)	Pending your answer: accept, or take a fix
Enter with the calendar open	First Enter picks the day, a second Enter saves	One Enter picks the day and saves	Pending your answer
Clicking a field's label	Did nothing	Puts the cursor in the field	Pending your answer (a small improvement)
"Saved" and "Cancelled" messages	Stayed until the mouse moved or a key was pressed, then closed about 3 s later	Close by themselves after 3 s. Same text and place (bottom right)	Pending your answer
After a server restart or outage	"Session Expired" or "Communication problem" messages	No message after a restart; the next click reloads the page and is lost. During an outage a striped bar "Connection lost, trying to reconnect..." shows, and the page reloads by itself when the server is back	Accepted at the first milestone
Other addresses	Any path showed the screen	Still any path shows the screen; a trailing slash is dropped from the address bar (`/foo/` shows as `/foo`). `/VAADIN/` answers 404 (old 403)	Accepted at the first milestone
Browser tab icon	Vaadin's default icon from the Valo theme	The browser's default icon (the app never had its own)	Parked as look only (section 6)

2.2 For operators

Area	Old	New
Framework	Vaadin 8.0.0.alpha2 (customer's original), then 8.14.5	`vaadin-core` 25.3.3 (Flow 25.3.4), Apache 2.0. No commercial components and no licence key needed
Servlet container	Jetty 9.2.3 (`jetty-maven-plugin`), `javax.servlet` 3.0	Jetty 12.1.14 `ee11` (`jetty-ee11-maven-plugin`), `jakarta.servlet` 6.1. Deploy on a Jakarta EE 11 / Servlet 6.1 container
Java	Java 8 bytecode	Java 21 (`release` 21); build and run on JDK 21
Production mode	Hard-coded off in the code; only the env and CI forced it on at start	Decided at build time: the war from `mvn package` always runs in production mode; `mvn jetty:run` is development mode

Area	Old	New
Dependencies	jsoup 1.14.3 (a known vulnerability), the unmaintained Vaadin 8 line with its warning banner, the `vaadin-prereleases` and `vaadin-addons` repositories	New runtime jars, all Apache 2.0: Jackson 3, Atmosphere, Reactor, collaboration-engine, Lumo and Aura; jsoup 1.23.2. Only Maven Central is used. commons-beanutils 1.9.2 unchanged
Logging	Jetty's and Vaadin 8's log lines	`slf4j-simple` 2.0.17: Vaadin's lines as `[thread] LEVEL logger - message` on stderr. Every start logs three `[WARNING] The XML schema [...] could not be found` lines from Jetty 12; they are harmless and the log check ignores them
Docker environment	Image `addressbook-old`, start about 2.5 s	Image `addressbook-new:<commit>` (about 1.4 GB), same `env/up.sh`, `reset.sh`, `down.sh`; every start and reset rebuilds the war offline and runs the unit tests first (about 5 s); a failing test stops that copy from starting (accepted)
Dev-mode tools	`?debug` window	Vaadin's dev tools and Copilot in development mode only; none of it is in the war. Usage statistics are switched off for every development run. Copilot checks its release notes at copilot.vaadin.com (only the Vaadin version is sent; accepted); when offline this logs an error in development mode

2.3 For developers

- **API:** every class uses Vaadin Flow (`com.vaadin.flow.*`) instead of the Vaadin 7 compatibility API: `Grid`, `FormLayout`, `Binder`, `DatePicker`, `Notification`. The four app classes keep their names and roles; `AppShell` is new and loads the Lumo theme.
- **Routes:** the screen is at `/` and at any other path (`@RouteAlias(":path*")`), as the old servlet mapping did.
- **Build:** `vaadin-maven-plugin` 25.3.3 adds a frontend step to every `mvn package` (it uses Vaadin's pre-built production bundle, so no Node run is needed). `maven-compiler-plugin` 3.14.1, surefire and failsafe 3.6.0 pinned. `vaadin-dev` is an optional dependency, so it is on the development classpath only.
- **Tests (new):** 7 JUnit 6 unit tests of the backend (storage returns copies; the 100 demo contacts match the original app's). 10 end-to-end tests in Playwright for Java 1.63.0, one per workflow that passed the old vs new comparison. They need a freshly started app, run in a fixed order, and add about 200 MB to every Maven repository and to the Docker image, because every build compiles them. See section 3.4.
- **CI:** one GitHub Actions job builds, starts the app in production mode, smoke-checks it, runs the end-to-end tests and fails on any error in the server log (section 3.5).

3 Environment, build, run, test and data

You need JDK 21 and Maven 3.9; Node 22 with npm for the CI steps; Docker for the reproducible environment. No Vaadin licence key and no database are needed (section 3.6). Full details are in `README.md`, `env/README.md` and `ci/README.md` in the repository.

3.1 Production build and run

```
mvn package          # production war in target/; runs the unit tests
mvn jetty:run-war    # builds and runs it on http://localhost:8080/
```

The log line "Vaadin is running in production mode." confirms the mode. To deploy elsewhere, put `target/addressbook-2.0.war` on a Jakarta EE 11 / Servlet 6.1 container. The app is a demo and has no production deployment target.

3.2 Development mode

```
mvn jetty:run        # development mode, http://localhost:8080/
```

Development mode adds Vaadin's dev tools. Usage statistics are off (set in `pom.xml`). It compiles the tests but does not run them.

3.3 Docker environment

```
env/up.sh            # build the image, start a copy on port 8080
env/reset.sh         # back to the seed contacts (restart)
env/down.sh          # stop and remove the copy
```

`APP_ENV_PORT`, `APP_ENV_PROJECT` and `APP_ENV_CONTAINER` run several copies side by side, each with its own data; `APP_ENV_PRODUCTION_MODE=false` runs development mode. The port is bound to 127.0.0.1 only. See `env/README.md`.

3.4 Tests

```
mvn test             # unit tests
mvn jetty:run-war    # or env/up.sh: a freshly started app
# then, in a second terminal, the end-to-end tests:
mvn verify -Pe2e -Dapp.url=http://localhost:8080
```

The contacts are in memory and the tests change them, so restart the app before each end-to-end run; a second run against the same app fails. Playwright downloads Chromium on the first run; set `PLAYWRIGHT_BROWSERS_PATH` and `PLAYWRIGHT_SKIP_BROWSER_DOWNLOAD=1` to use browsers you already have. Failure screenshots go to `target/e2e-failures/`.

The workflow "Enter a birth date with the calendar and by typing" has no end-to-end test yet: it waits for the decision in section 7.

3.5 CI

`.github/workflows/ci.yml` runs one job on `ubuntu-24.04` on every push to `main`, `mig-dev`, `stage/**`, `stabilise/**` and `fix/**` and on every pull request. It needs no secrets. In order: JDK 21, `mvn -B package`, Node 22 and Playwright with Chromium, `ci/start-app.sh` (production mode), the smoke check (the contact list must show rows, with no console or page errors), the production-mode check, the end-to-end tests, the server-log check (any `SEVERE` or `ERROR` line or stack trace fails the job), and stop. On failure only, logs and screenshots are kept for 3 days. A run takes a few minutes. `ci/README.md` lists the same steps to run locally.

Playwright is pinned twice: in `pom.xml` (the end-to-end tests) and in `ci/package-lock.json` (the smoke check and the browser install). The tests use the browser that CI installs, so the two versions must be changed together.

3.6 Database

None. The app has no database. Its 100 demo contacts are generated in memory at start from a fixed seed and are shared by every user until the app restarts. There are no migration scripts.

4 Known pre-existing issues

These issues were in the old app before the migration. The migration kept them on purpose, so that the new app behaves the same; none was introduced by the migration. These are the ones a user or developer is likely to notice. The full list (36 items, including build and tooling details) is available on request.

Users notice:

- All users share one contact list: one user's changes are visible to everyone until the app restarts. Another user's changes only appear after your own save, filter or reload. When two users edit the same contact, the last save wins without a warning.
- There is no Delete button; a contact can only be removed by restarting the app (which resets all data).
- There are no input checks: names can be empty and email and phone take any text, so an empty new contact saves as a blank row.
- When a birth date is not a valid date, Save does nothing and gives no message about the save (the new app does show "Date format not recognized" at the field; section 2.1).
- Cancel on a new contact shows "Cancelled" but leaves the form open with what was typed.
- The filter searches the whole contact record, not only the shown columns: it also matches the phone number, the birth date in English text and internal field names (typing `name` matches every contact).
- Enter saves the open contact even when the cursor is in the filter field.
- Birth dates show two-digit years, and future birth dates are accepted.

Developers notice:

- commons-beanutils 1.9.2 (CVE-2019-10086, CVE-2025-48734) and, through it, commons-collections 3.2.1 (CVE-2015-6420 / CVE-2015-7501). Kept at the agreed version; the app copies only its own beans with it and deserialises nothing.
- The contact service is a lazily created singleton that is not thread-safe at start-up, and it is not serialisable, so sessions cannot be persisted or clustered.
- CI and env scripts: GitHub Actions pinned to major tags instead of commit hashes; `ci/start-app.sh` accepts any server answering on its port; the docs do not list `curl`, `sha256sum`, `find` and `xargs` as prerequisites; env copies run without a memory limit.

5 Old and new side by side

The 11 user workflows recorded on the old app were run on the new app step by step and compared screenshot by screenshot. The walkthrough shipped next to this document shows each workflow with the old and new screenshots of every step and a note on each difference: docs/walkthrough/index.html (open it in a browser; it needs no server).

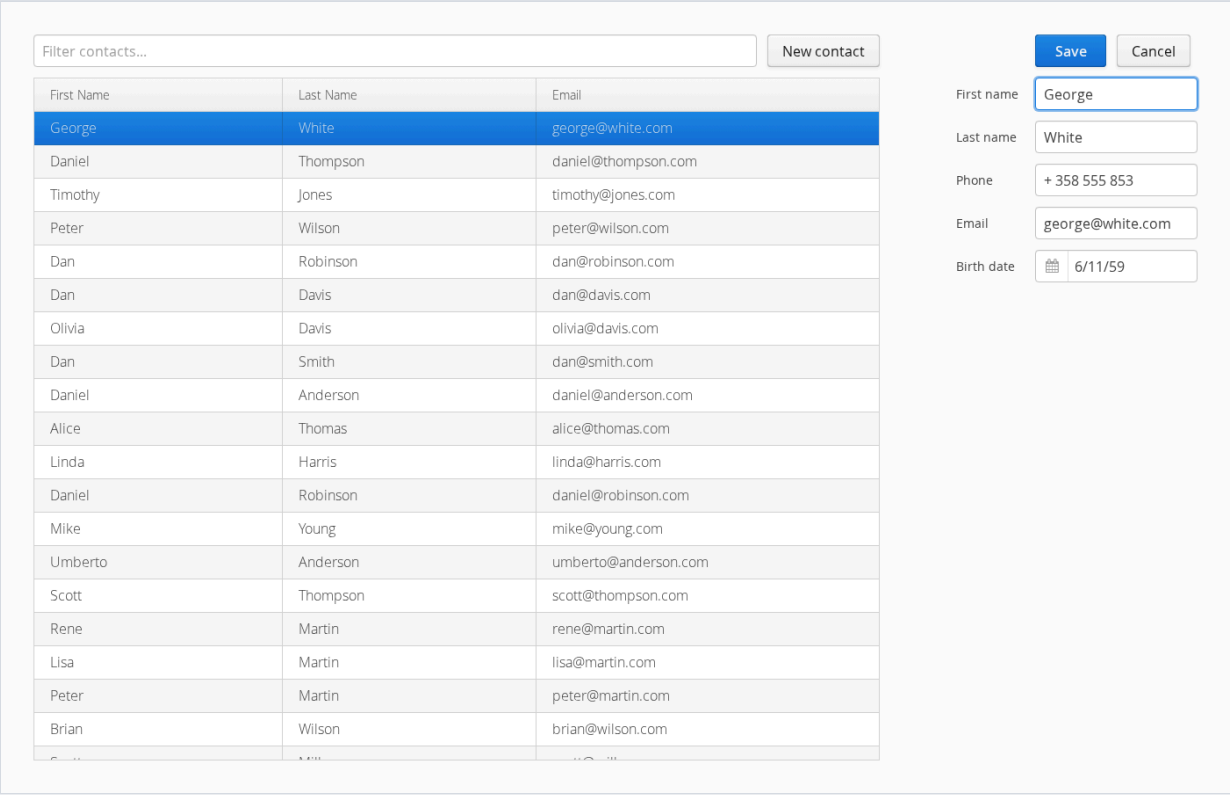


Figure 2. The old app (Vaadin 8.14.5, Valo) with the same contact open, 1400 x 900 window. Compare with figure 1.

Workflow	Result
Browse the contact list; filter it; open and close a contact; add, edit, cancel; save with missing values; see another user's changes	Same behaviour. About one more row is visible with Lumo (accepted with the theme)
Try to save an invalid birth date	Same result; the message shows under the field (pending your answer)
Two users edit the same contact	Same result; the "Saved" message closes by itself (pending your answer)
Enter a birth date with the calendar and by typing	Differs in one step: the calendar for a new contact opens on the last month shown (pending your answer)

6 Possible follow-up work

Improvements that are not needed for the migration and were left out on purpose. Each is small. They are input for a follow-up, not part of this delivery.

Look (the migration aimed at the same structure, not the same pixels; checked at 1400, 1024 and 1920 px wide unless noted):

- Lumo differs from Valo by design: grey-filled fields, borderless buttons, a taller grid header (56 px, old 38), pale row selection, no column lines or zebra rows, a pink fill for an invalid field, system font, white background, different sort and calendar icons (checked at 1400 px). A restoration step can bring the old look back where wanted.
- Save and Cancel sit 12 px lower, the grid starts 12 px lower, the gap between list and form is 16 px (old 12 px), form fields are 192 px wide (old 185 px).
- Notification text is dark on white (old blue). No tab icon: add a `favicon.ico` if wanted.

CI speed:

- Cache the Chromium install, or use a runner image that has it (one early run spent 9.5 minutes installing it; usually about 15 s).
- Run the end-to-end tests without rebuilding the war while the app is running.
- Let CI start the app through `env/up.sh`, so there is one way to start it instead of two.

Tests:

- Replace the fixed waits in the end-to-end tests and the wait on Vaadin's internal client object with waits on what the page shows.
- Give the demo-data unit test its own contact service, so test order cannot matter later.
- Add the end-to-end test for the birth-date workflow once section 7 is decided.

Dependencies and code:

- Replace the bean copy by a small copy method so commons-beanutils and commons-collections, with their known vulnerabilities, can be removed.
- Store the birth date as a date without time (`LocalDate`) instead of converting in the form; this removes the time-zone and time-of-day handling.
- Run the war on a slim Jetty image instead of Maven and a JDK in the Docker image (smaller and faster to pull).

7 Open decisions

7.1 Behaviour differences to accept or reject

Each item is pending your answer. Where the migration team has recorded a recommendation, it is given.

#	Difference	Options	Recommendation
1	Calendar for a new contact opens on the month last shown in the tab, not on the current month	A: accept the difference. B: take a prepared fix, which works by setting an internal value of Vaadin's date picker from JavaScript; it is tested, but a Vaadin update can break it silently in the app (the workflow test would catch it)	A, accept: a small difference that comes from Vaadin's own component
2	A typed two-digit year in the boundary year is read by year, not by day	Accept, or ask for the old reading	Accept: framework behaviour
3	The date order is always the US one (month/day/year), also for non-US English browsers	Accept, or ask for the browser's date order	Accept: the users are US English
4	Clicking a field's label puts the cursor in the field	Accept, or ask to remove it	Accept: a small improvement
5	The calendar popup has a new layout (scrolling months, year column, Today and Cancel)	Accept, or ask for a restyle	Accept: look only, same function
6	"Saved" and "Cancelled" close by themselves after 3 s	Accept, or ask for a longer time or a close button	Pending your answer
7	"Date format not recognized" shows under the field instead of as a tooltip	Accept, or ask for the old marker	Pending your answer
8	With the calendar open, one Enter picks the day and saves	Accept, or ask for the old two-step Enter	Pending your answer

Until these are answered, the workflows "Enter a birth date", "Try to save an invalid birth date" and "Two users edit the same contact" are counted as not yet accepted.

7.2 Where the migrated code ends up

The output repository has the stabilised Vaadin 8 app on `main` and the migrated Vaadin 25 app on `mig-dev`. Pending your answer: merge `mig-dev` into `main`, replace `main` with it, or keep both branches side by side. If the repository exists so that the old and new versions can be compared, keeping both side by side is the simplest.

7.3 Acceptance

After these answers, the remaining steps are a walkthrough of this document with you and your acceptance of the migration.